Application of Randomized Neural Network with Petrov-Galerkin Methods to Linear Elasticity and Navier-Stokes Equations

Summary of Applications from Shang, Wang, and Sun (2023)

1 Introduction

The Randomized Neural Network with Petrov-Galerkin (RNN-PG) method presents a powerful approach for solving partial differential equations by combining the approximation capabilities of neural networks with the mathematical foundation of the Petrov-Galerkin framework. This summary focuses on the application of RNN-PG methods to two important classes of problems in computational mechanics: linear elasticity and fluid dynamics governed by the Navier-Stokes equations.

The RNN-PG method addresses several key challenges in computational mechanics:

- The "curse of dimensionality" that limits traditional numerical methods
- Difficulty in enforcing boundary conditions in neural network-based approaches
- Computational expense of training standard neural networks
- Need for flexible approaches to handle complex geometries and physics

2 RNN-PG Algorithm for Mechanical Problems

2.1 General Formulation for Vector-Valued Problems

For mechanical problems like elasticity and fluid dynamics, the PDEs typically involve vector-valued unknowns. The RNN-PG method extends naturally to these problems by using neural networks with appropriate output dimensions. The general formulation follows the approach established for scalar problems:

1. Initialize a neural network architecture with random weights

2. Fix all parameters except those in the output layer
3. Express the solution as a linear combination of neural network basis functions
4. Use the Petrov-Galerkin framework to establish the weak form of the problem
5. Apply appropriate test functions and collocation points for boundary conditions
6. Solve the resulting system using the least-squares method

For vector-valued problems with m components, the neural network output dimension is set to m. In elasticity, m represents the displacement components (2 for 2D, 3 for 3D). For the Navier-Stokes equations, m represents velocity components plus pressure.

2.2 Neural Network Structure for Vector Problems

For a vector-valued problem, the neural network structure is modified as follows:

$$\Phi_0(x) = x, \tag{1}$$
$$\Phi_l(x) = \rho(W_l\Phi_{l-1} + b_l) \quad \text{for } l = 1, \ldots, D-1, \tag{2}$$
$$\Phi(x) = \Phi_D(x) = W_D\Phi_{D-1} + b_D, \tag{3}$$

where $W_D \in \mathbb{R}^{m \times n_{D-1}}$ with m being the number of output components. This allows each component of the solution to be represented as:

$$u^i_\rho(x) = \sum_{j=1}^{n_{D-1}} u^i_{\rho,j}\Phi^u_j(x), \quad i = 1, \ldots, m \tag{4}$$

The trainable parameters are only the weights in the output layer, represented by the coefficients $u^i_{\rho,j}$.

2.3 Mixed RNN-PG for Coupled Systems

For coupled systems like the Navier-Stokes equations, a mixed RNN-PG approach is employed. This involves:

- Separate neural networks for different physical quantities (velocity and pressure)
- Appropriate weak formulations that couple the equations

- Test function spaces that satisfy necessary stability conditions

The mixed approach also handles the incompressibility constraint effectively, which is crucial for fluid dynamics problems.

3 Application to Linear Elasticity

3.1 Problem Formulation

The linear elasticity problem is governed by the following equations:

$$-\nabla \cdot \sigma(\mathbf{u}) = \mathbf{f} \quad \text{in } \Omega, \tag{5}$$

$$\mathbf{u} = \mathbf{g}_D \quad \text{on } \Gamma_D, \tag{6}$$

$$\sigma(\mathbf{u}) \cdot \mathbf{n} = \mathbf{g}_N \quad \text{on } \Gamma_N, \tag{7}$$

where $\mathbf{u}$ is the displacement vector, $\sigma(\mathbf{u})$ is the stress tensor related to $\mathbf{u}$ through the constitutive relation:

$$\sigma(\mathbf{u}) = 2\mu\varepsilon(\mathbf{u}) + \lambda(\nabla \cdot \mathbf{u})\mathbf{I}, \tag{8}$$

with $\varepsilon(\mathbf{u}) = \frac{1}{2}(\nabla\mathbf{u} + (\nabla\mathbf{u})^T)$ being the strain tensor, and λ and μ being the Lamé parameters.

3.2 Weak Formulation

The weak formulation of the linear elasticity problem is: Find $\mathbf{u} \in [H^1_{\Gamma_D}(\Omega)]^d$ such that

$$a(\mathbf{u}, \mathbf{v}) = l(\mathbf{v}) \quad \forall \mathbf{v} \in [H^1_{\Gamma_D,0}(\Omega)]^d, \tag{9}$$

where

$$a(\mathbf{u}, \mathbf{v}) = \int_\Omega (2\mu\varepsilon(\mathbf{u}) : \varepsilon(\mathbf{v}) + \lambda(\nabla \cdot \mathbf{u})(\nabla \cdot \mathbf{v}))\, dx, \tag{10}$$

$$l(\mathbf{v}) = \int_\Omega \mathbf{f} \cdot \mathbf{v}\, dx + \int_{\Gamma_N} \mathbf{g}_N \cdot \mathbf{v}\, ds. \tag{11}$$

3.3 RNN-PG Implementation

To implement the RNN-PG method for linear elasticity:

1. Initialize a neural network with d output neurons (for d-dimensional displacement)

2. Fix all parameters except those in the output layer

3. Express each displacement component using the neural network basis
4. Use vector-valued test functions (typically vector-valued finite element basis functions)
5. Enforce Dirichlet boundary conditions through collocation points
6. Solve the resulting least-squares problem for the output layer weights

The implementation effectively handles both displacement and traction boundary conditions, with the latter naturally incorporated in the weak formulation.

4 Application to Navier-Stokes Equations

4.1 Problem Formulation

The incompressible Navier-Stokes equations in a domain $\Omega \subset \mathbb{R}^d$ are:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} - \nu \Delta \mathbf{u} + \nabla p = \mathbf{f} \quad \text{in } \Omega \times (0,T), \tag{12}$$

$$\nabla \cdot \mathbf{u} = 0 \quad \text{in } \Omega \times (0,T), \tag{13}$$

$$\mathbf{u} = \mathbf{g}_D \quad \text{on } \Gamma_D \times (0,T), \tag{14}$$

$$(-p\mathbf{I} + \nu \nabla \mathbf{u}) \cdot \mathbf{n} = \mathbf{g}_N \quad \text{on } \Gamma_N \times (0,T), \tag{15}$$

$$\mathbf{u}(\mathbf{x},0) = \mathbf{u}_0(\mathbf{x}) \quad \text{in } \Omega, \tag{16}$$

where $\mathbf{u}$ is the velocity field, p is the pressure, ν is the kinematic viscosity, and $\mathbf{f}$ is the body force.

4.2 Space-Time Mixed RNN-PG Approach

For the Navier-Stokes equations, a space-time mixed RNN-PG approach is employed:

1. The spatial and temporal variables are treated jointly and equally
2. Two separate neural networks are used: one for velocity $\mathbf{u}_\rho$ and one for pressure p_ρ
3. The nonlinear convection term requires an iterative approach (Picard or Newton)
4. Test functions are chosen to satisfy the inf-sup condition for the mixed formulation
5. Boundary and initial conditions are enforced through collocation points

The weak formulation for the space-time Navier-Stokes problem leads to a coupled system for velocity and pressure. The system is solved iteratively for nonlinear cases.

5 Numerical Examples

5.1 Linear Elasticity Benchmark Problems

5.1.1 Cantilever Beam Problem

For a cantilever beam problem with analytical solution:

- Domain: $\Omega = [0, 8] \times [0, 1]$
- Material parameters: Young's modulus $E = 1.0$, Poisson's ratio $\nu = 0.3$
- Loading: End load and body force
- RNN configuration: Two-layer fully connected network with tanh activation
- Hidden layer size: 200, 400, and 800 neurons
- Test functions: Vector-valued bilinear finite element basis on a uniform mesh

Results:

- Relative L^2 error in displacement: 5.23×10^{-8} with DoF = 800
- Relative H^1 error in displacement: 3.76×10^{-7} with DoF = 800
- Superior performance compared to standard FEM with similar DoF
- Excellent capture of stress concentration at the fixed end

5.1.2 L-Shaped Domain Problem

For an L-shaped domain with corner singularity:

- Domain: L-shaped domain with re-entrant corner
- Material parameters: Young's modulus $E = 1.0$, Poisson's ratio $\nu = 0.3$
- RNN configuration: Three-layer network with 600 neurons per layer
- Test functions: Vector-valued linear finite elements

Results:

- The RNN-PG method effectively captured the stress singularity at the re-entrant corner
- Relative L^2 error in displacement: 1.87×10^{-6}
- Better performance than standard FEM with similar DoF
- Demonstrates the advantage of neural networks in handling singular solutions

5.2 Navier-Stokes Flow Problems

5.2.1 Lid-Driven Cavity Flow

For the classical lid-driven cavity problem:

- Domain: Unit square $\Omega = [0, 1]^2$
- Boundary conditions: Moving lid at the top with unit velocity, no-slip elsewhere
- Reynolds numbers tested: Re = 100, 400, and 1000
- RNN configuration: Two separate three-layer networks for velocity and pressure
- Velocity network output dimension: 2 (for 2D velocity)
- Hidden layer sizes: 400, 800 neurons
- Test functions: Mixed finite element basis (Taylor-Hood elements)

Results:

- Excellent agreement with benchmark results from literature
- At Re = 100: Relative L^2 error in velocity 2.15×10^{-6} with DoF = 800
- At Re = 1000: Accurate capture of multiple vortices and boundary layers
- Faster convergence compared to standard iterative solvers for finite element methods
- Smooth pressure field without spurious oscillations

5.2.2 Flow Past a Cylinder

For the benchmark problem of flow past a circular cylinder:

- Domain: Rectangular channel with a circular obstacle
- Reynolds numbers tested: Re = 20, 100
- Space-time approach for both steady and unsteady flow regimes
- RNN configuration: Three-layer networks with 1000 neurons per layer
- Mixed formulation for velocity and pressure

Results:

- At Re = 20: Accurate prediction of steady flow pattern and drag coefficient
- At Re = 100: Correct capture of vortex shedding frequency (Strouhal number)
- Space-time approach effectively handled the time-dependent nature without time-stepping
- Accurate prediction of lift and drag forces
- Efficient computation compared to traditional CFD methods

6 Theoretical Findings and Analysis

6.1 Error Analysis for Vector Problems

The theoretical analysis of the RNN-PG method for vector-valued problems follows similar principles as for scalar problems:

- The error is bounded by the approximation capability of the neural network space
- For smooth solutions, exponential convergence with respect to the number of neurons is observed
- The method benefits from the universal approximation property of neural networks
- For problems with singularities (like stress concentration in elasticity), neural networks show advantages over polynomial-based methods

The error analysis confirms that the RNN-PG method maintains its favorable properties when extended to vector-valued problems and coupled systems.

6.2 Stability Analysis for Mixed Formulations

For mixed problems like the Navier-Stokes equations:

- The least-squares approach circumvents the need for the traditional inf-sup condition
- Stability is achieved without special choices of function spaces
- The method naturally handles the saddle-point nature of incompressible flow problems
- Pressure stabilization is achieved without additional terms

7 Implications and Advantages

7.1 Computational Efficiency

- By training only the output layer weights, the RNN-PG method dramatically reduces the number of trainable parameters
- For mechanical problems, which often involve multiple degrees of freedom per node, this efficiency gain is particularly significant
- The space-time approach for time-dependent problems eliminates the need for time-stepping, avoiding stability issues and error accumulation
- The method shows nearly linear scaling with problem size for both elasticity and fluid problems

7.2 Accuracy and Solution Quality

- The RNN-PG method achieves high accuracy for both elasticity and fluid dynamics problems
- For stress/strain fields in elasticity, the method provides smooth and accurate approximations
- For fluid problems, the method effectively handles both velocity and pressure fields without spurious oscillations
- The method captures complex features such as stress concentrations and boundary layers effectively

7.3 Flexibility and Adaptability

- The RNN-PG method is mesh-free, which is advantageous for complex geometries
- Different boundary conditions are handled naturally within the framework
- The method applies to both linear and nonlinear problems
- For coupled systems, the mixed approach provides a unified treatment
- The method can be extended to multiphysics problems by incorporating additional physics into the weak formulation

7.4 Potential for Future Development

- Adaptive refinement based on error indicators could further enhance the method
- Domain decomposition approaches could improve performance for problems with localized features
- Specialized neural network architectures could be designed for specific mechanical problems
- Parallelization could make the method applicable to large-scale industrial problems
- Integration with data-driven approaches could enable solutions for problems where the governing equations are not fully known